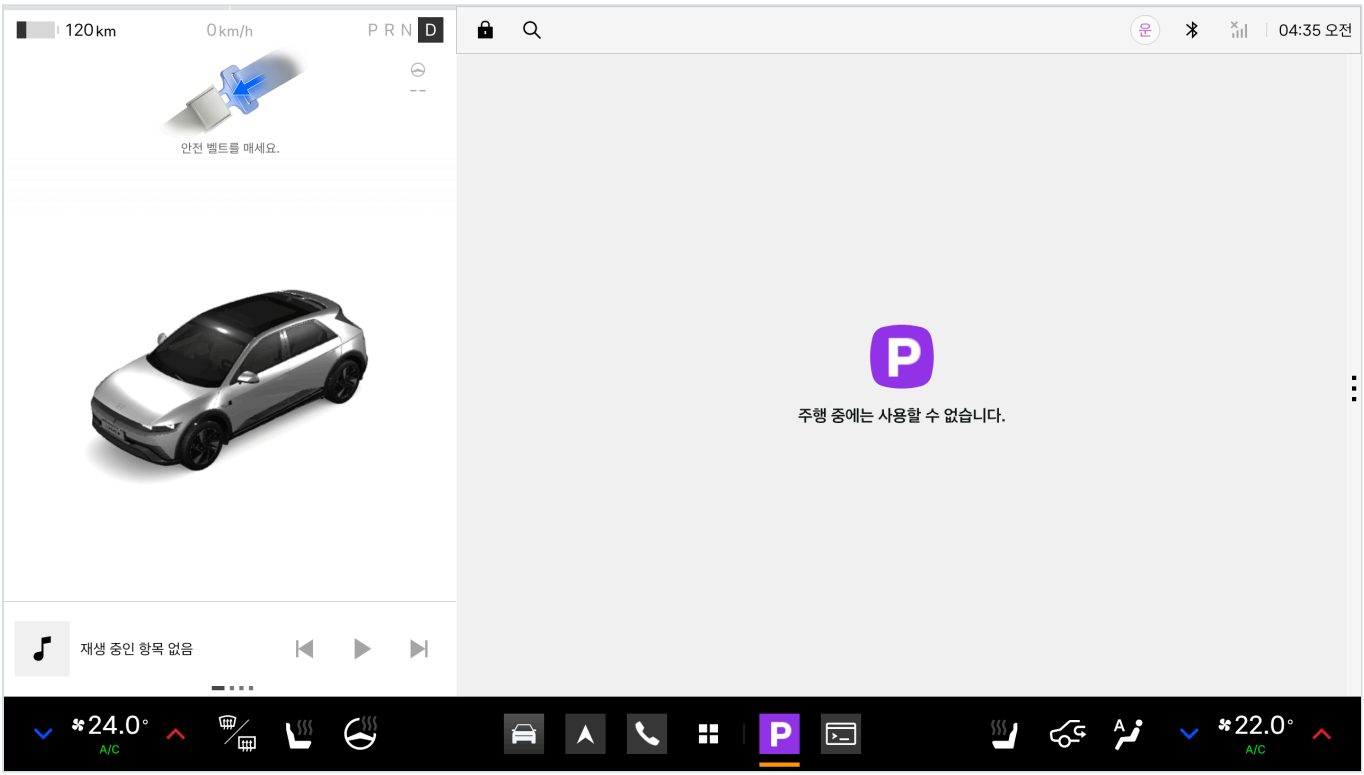
# Pleos Connect 에뮬레이터로 무엇을 만들고 검증할까

주차와 주행을 직접 전환하고,
차량 상태 변경이 화면에 반영되는 과정을 확인했다.

실험일 2026년 11월 2일 · Connect 2.1.0 · Android 14 ARM64
Vehicle SDK 2.3.2 / Vehicle Service 2.3.22




실제 캡처. D·0km/h에서도 일반 앱의 사용 제한 화면을 확인했다. 왼쪽의 120km는 별도로 주입한 잔여 주행거리다.

## 팀이 먼저 알아야 할 세 가지

1. **정지와 주차는 다르다.** 이 이미지에서는 D/R/N·0km/h에도 UX 제한이 켜졌고, P 복귀 시 해제됐다.
2. **기능 시연을 위한 상태 주입이 가능하다.** 공조, 도어·창문, 배터리·잔여거리의 화면 반영을 직접 확인했다.
3. **콘솔의 제어 권한은 우리 앱의 권한이 아니다.** SDK 지원 여부·권한·배포 승인은 별도로 확인해야 한다.

# 어디까지 직접 확인했나

공식 Virtual Console의 조작부와 실제 Pleos 화면을 함께 사용했다. 기어·가속, 공조 설정, 차량 편의 장치, 에너지 상태를 변경하고 스크린샷과 Car Service 진단값을 대조했다.

| 항목 | 이번 실험의 실제 조건 |
|---|---|
| 호스트 / 에뮬레이터 | Apple Silicon Mac · Android Emulator 37.2.13 |
| 차량 이미지 / AVD | Pleos Connect 2.1.0 · `Pleos_Connect_2_1_0` |
| 게스트 / 화면 / 메모리 | Android 14, API 34, arm64-v8a · 2560×1440 · RAM 4GB |
| 앱 / 차량 서비스 | `dev.pleos.starter` · Vehicle Service 2.3.22 |
| 앱 구현 상태 | 기본 Activity, DO 선언 없음. Vehicle SDK 의존성만 포함하며 차량 API 호출 코드는 없음. |
| 진단상 정책 범위 | `android.car.drivingsafetyregion.all`<br>한국어 UI를 특정 국가의 실차 규제와 동일시하지 않음. |

직접 확인

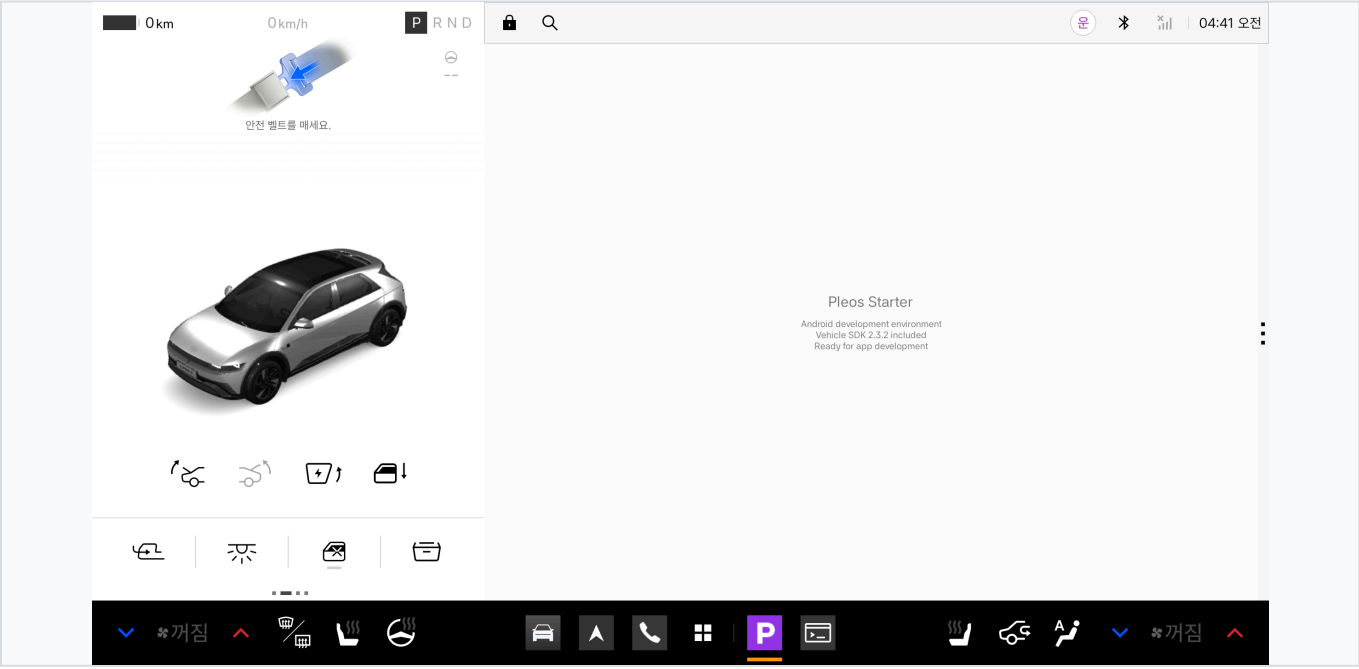
조작 후 콘솔·차량 화면 또는 진단값에서 변화가 확인된 내용.

문서 기준

공식 문서에 명시됐으나 이번 실험에서 끝까지 실행하지 않은 내용.

**일반 앱의 SDK 연동 성공까지 증명한 것은 아니다.**

이번에는 플랫폼과 기본 차량 UI의 반응을 확인했다. 실제 개발 앱의 capability 검사, 권한 처리, API 응답·오류 처리는 다음 개발 단계의 검증 대상이다.

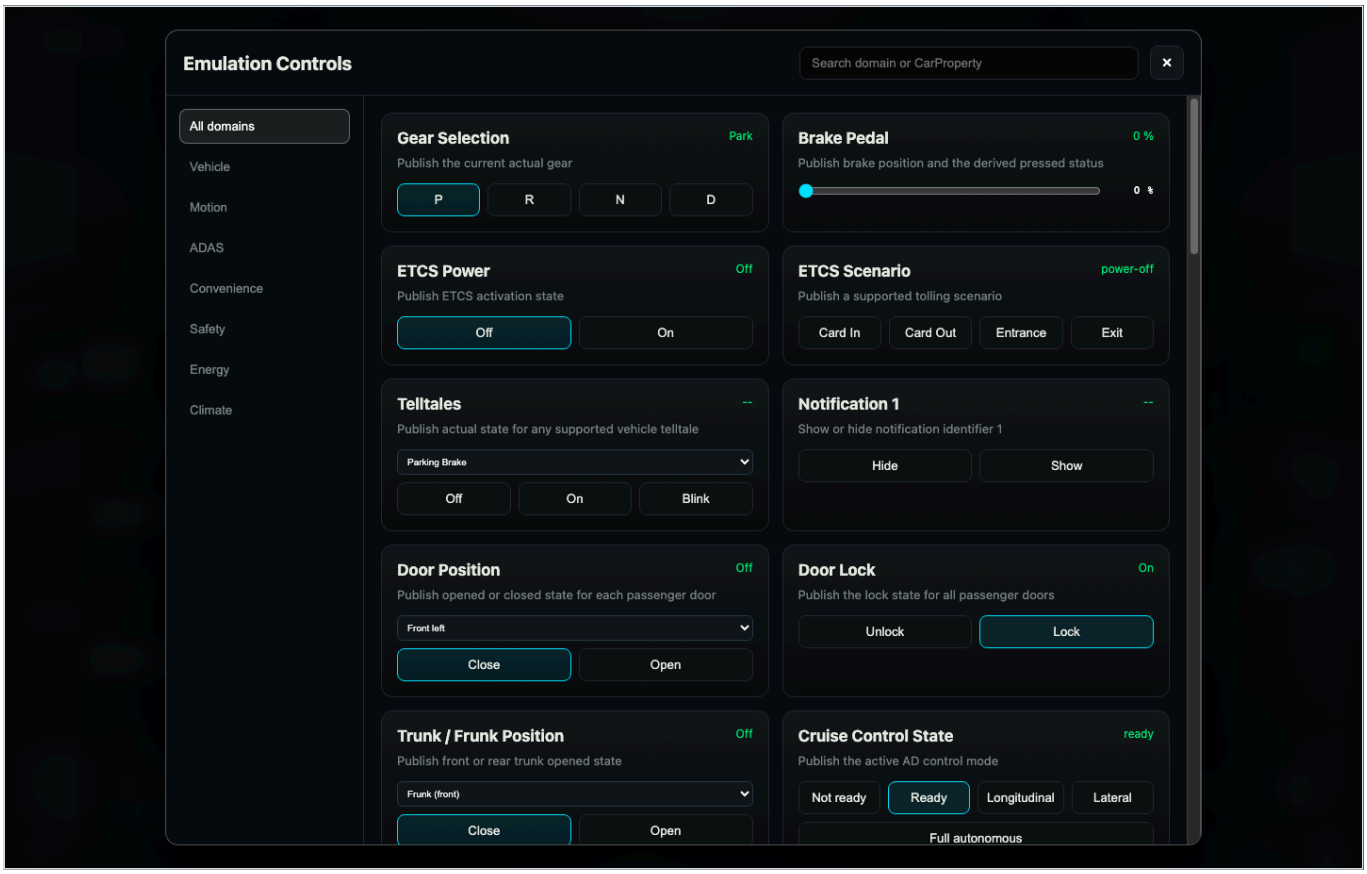



실험 종료 후 P·0km/h, 도어·창문 닫힘, 공조 Off로 복원했다. 경고 확인 창도 직접 닫아 기본 앱 화면을 확인했다.

# 구현 아이디어를 시험할 수 있는 영역

아래는 **에뮬레이터의 입력·상태 주입 범위**다. 앱에서 같은 기능을 읽거나 제어할 수 있는지는 SDK API와 권한을 따로 확인한다. [S1, S2, S5]

| 도메인 | 콘솔 제공 항목의 예 | 이번 확인 수준 |
|---|---|---|
| Vehicle | 차량 알림 표시·숨김 | 카드 확인 |
| Motion | P/R/N/D, 브레이크, 조향각, 누적 주행거리, 핸들 열선 | 기어·가속·속도 직접 조작 |
| ADAS | 크루즈 제어 상태, 목표 속도 | 제공 항목 확인 |
| Convenience | 도어·잠금·프렁크·창문, 조명, 와이퍼, 좌석 탑승, ETCS | 도어·프렁크·창문 직접 조작 |
| Safety | 안전벨트·ABS·TPMS 등 경고등 Off / On / Blink | 카드 확인; 프렁크 경고 별도 관찰 |
| Energy | SOC, 잔여거리, 배터리 에너지·용량, 충전 한도·포트 | SOC·잔여거리 직접 조작 |
| Climate | 공조 전원, 온도, 풍량·방향, 열선·통풍, 내기순환·제상 | 전원·온도·풍량 직접 조작 |




실제 Emulation Controls 패널. 카드 제목·영역 선택·설정값을 확인할 수 있으며, 전체 목록은 스크롤된다.

ADAS의 “Full autonomous”는 주입 가능한 상태 값의 이름이다. 실제 자율주행 알고리즘·센서 인지·차량 제어 성능을 검증했다는 뜻은 아니다.

# 차량 상태를 바꾸는 실험 도구

에뮬레이터가 실행된 상태에서 8088 포트를 전달하면 PC 브라우저로 콘솔에 접속할 수 있다. 이번 실험에서는 **DDS ONLINE**과 AVD 연결 상태를 확인한 뒤 조작했다. [S1, S2]

```
adb -s emulator-5554 forward tcp:8088 tcp:8088
# PC 브라우저에서 접속
http://127.0.0.1:8088/
```




직접 확인한 Forest 테마. PRND, 가속·브레이크, 조향, 공조·차체 제어가 한 화면에 있다. 상단 도로 그림은 차량 기능의 시험용 시각화다.

### 1. 상태를 만든다

기어를 선택하고 가속·브레이크 또는 도메인별 카드 값을 조작한다.

### 2. 반영을 확인한다

콘솔의 입력값만 보지 말고 차량 표시와 앱의 응답·진단값도 함께 읽는다.

### 3. 복귀까지 시험한다

정지와 P 복귀, 구독 해제, UI 복원까지 한 시나리오로 검증한다.

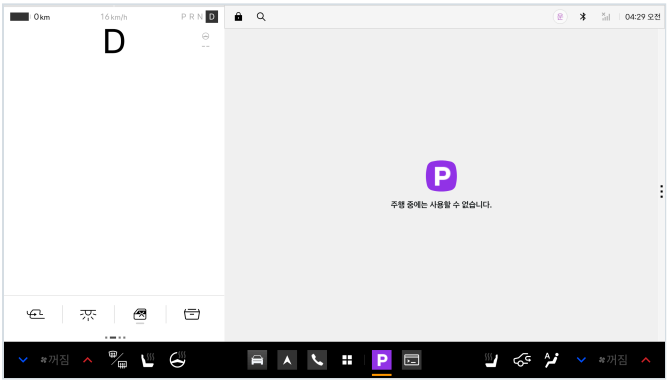
> **브라우저 그림과 차량 신호는 분리해서 본다.**
> 이번 테스트 브라우저에서는 Highway의 WebGL 2 시각화가 실패했지만, 상태 주입과 DDS 연결은 동작했다. Forest는 정상 표시됐다. 테마 전환 자체가 앱 권한이나 주행 규제를 바꾸지는 않는다. [S2]

# “속도 0”만으로 비주행을 판단하면 안 된다

**DO(Distraction Optimized)**는 주행 중 사용을 고려해 최적화된 앱·Activity를 뜻한다. 아래의 “DO 요구”는 앱의 선언 여부가 아니라 플랫폼이 요구하는 상태다.

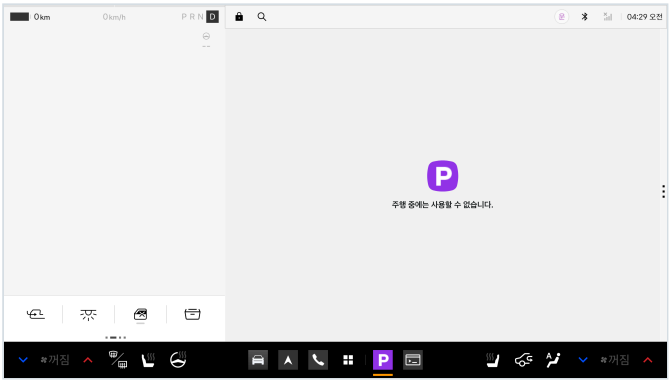
| 시험 조건 | 주행 상태 진단값 | DO 요구 / 제한 | 확인한 결과 |
|---|---|---|---|
| P · 0km/h | 0 · parked | false / `0x0` | 일반 앱 정상 표시 |
| D · 0km/h | 1 · idling | true / `0x50` | UX 제한 상태 진입 |
| D · 가속 후 이동 | 2 · moving | true / `0x50` | 일반 앱 사용 불가 화면 |
| D · 다시 0km/h | 1 · idling | true / `0x50` | 사용 불가 화면 유지 |
| R / N · 0km/h | 각각 1 · idling | true / `0x50` | 진단값으로 제한 유지 확인 |
| P로 복귀 | 0 · parked | false / `0x0` | 제한 해제·일반 앱 복원 |

이동 중




30km/h까지 가속한 뒤 캡처. 화면을 찍을 때의 실제 표시 속도는 16km/h였다.

D에서 정지




속도가 0으로 내려가도 D 상태에서는 사용 제한 화면이 유지됐다.

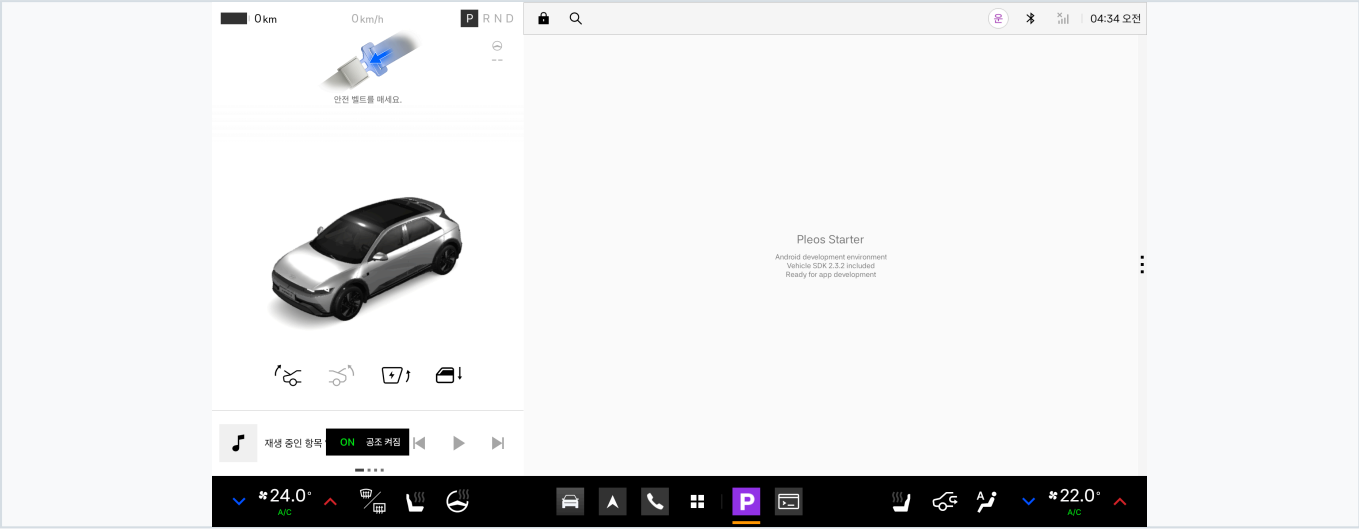
## 진단값은 이렇게 해석했다

```
P:   Current Driving State: 0
     Display id: 0 UXR: DO: false UxR: 0
D:   Current Driving State: 1 또는 2
     Display id: 0 UXR: DO: true UxR: 80
```

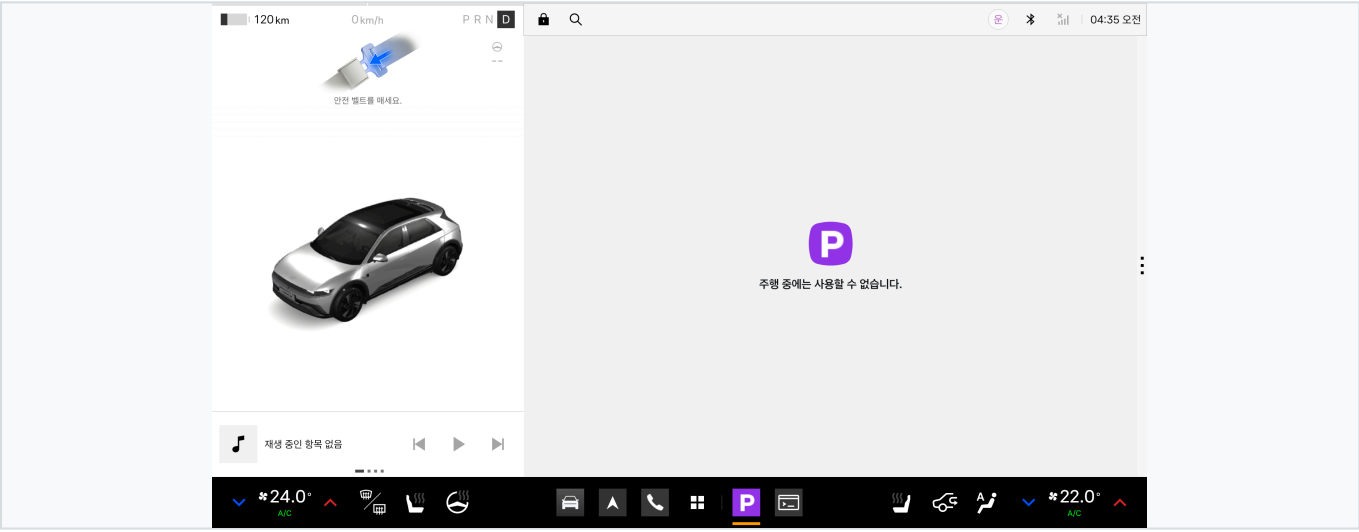
십진수 `80`은 `0x50`이다. 설치된 Android API 34의 공개 상수와 대조하면 `NO_VIDEO(16) | NO_SETUP(64)`에 해당한다. **앱 전체의 DO 적합성 판단**과 **앱 내부 기능별 제한**은 별도로 처리해야 한다. [S3, S4]

> **이 표는 모든 차종·국가의 규제표가 아니다.**
> 현재 이미지의 정책 범위는 `all`이다. 배포 대상에서는 지역·차량·플랫폼별 상태와 실제 `CarUxRestrictions` 값을 다시 확인해야 한다.

# 상태 기반 기능을 시연하기 좋은 영역




**공조:** 전원 On, 운전석 24°C, 풍량 5로 변경했다. 콘솔에서 세 값을 확인했고, 차량 하단에는 운전석 24.0°와 A/C 상태가 표시됐다.

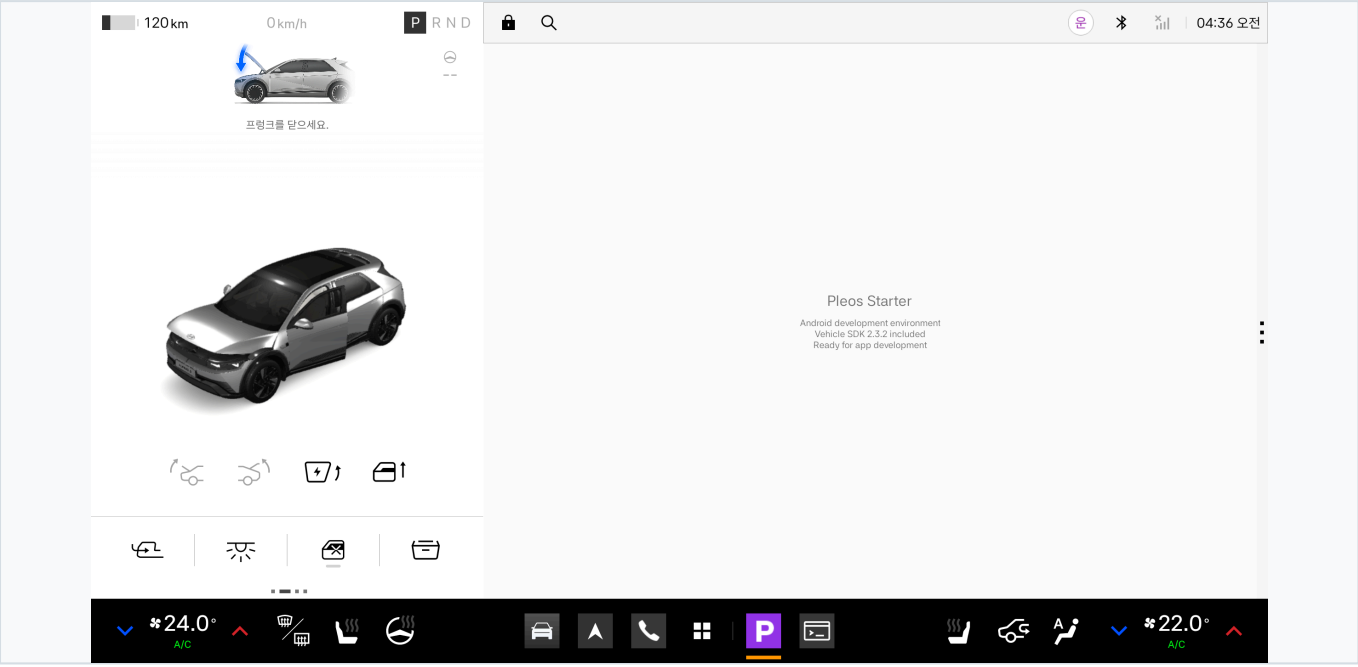



**에너지:** SOC 25%, 잔여거리 120,000m를 설정했다. 차량 계기에는 배터리 잔량 도형과 120km가 반영됐다. 콘솔은 m, 차량 화면은 km로 표시한다.

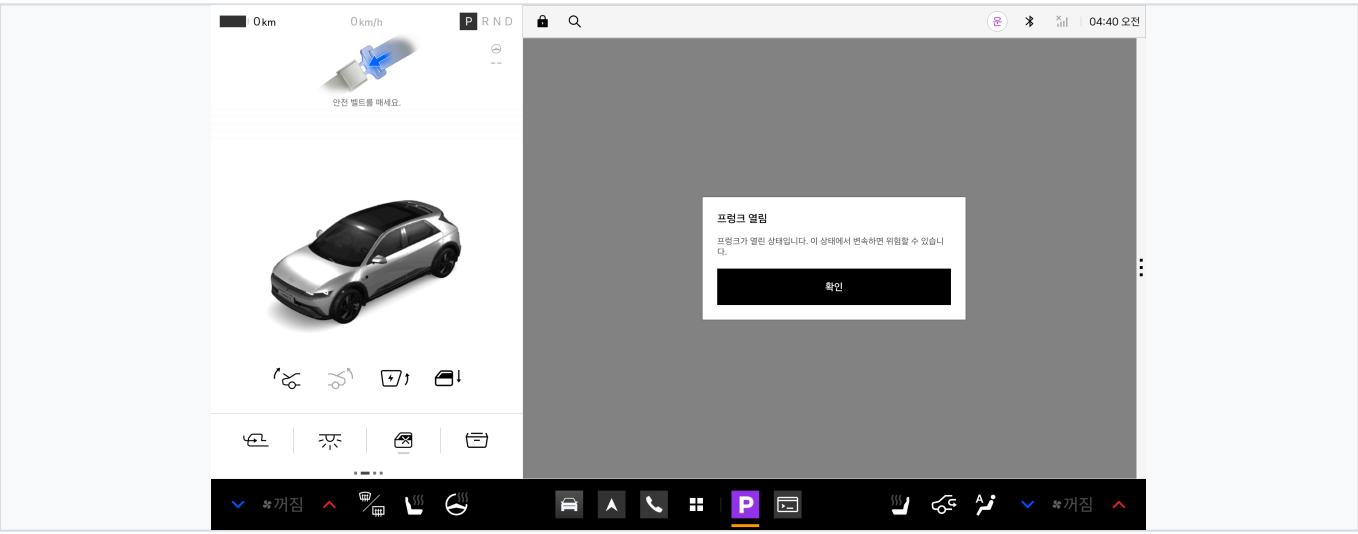
| 구현 아이디어 | 이번 환경에서 시험할 수 있는 것 |
|---|---|
| 전기차 상태 대시보드 | SOC·잔여거리 변화에 따른 카드, 경고, 추천 조건의 표시 |
| 출발 전 실내 환경 점검 | 공조 전원·목표 온도·풍량 상태에 따른 안내 UI |
| 충전·휴식 안내 | 주입한 에너지 상태를 입력으로 하는 조건 분기와 메시지 |

> **이번 실험은 상태 주입이다.**
>
> SOC와 잔여거리는 각각 설정했다. 실제 소비전력·기온·주행 패턴으로 잔여거리를 계산하는 물리 모델이나 배터리 성능을 검증한 것은 아니다. 자동 공조 제어 앱은 별도의 제어 권한 검증이 필요하다.

# 차량 상태가 경고와 3D 표시에 연결된다




앞왼쪽 도어 Open, 앞왼쪽 창문 50%, 프렁크 Open을 설정했다. 콘솔에서 세 값을 확인했고, 차량 화면의 도어 그림과 “프렁크를 닫으세요” 안내가 바뀌었다.




프렁크가 열린 상태에서 기어 전환을 시험하자 경고 창도 나타났다. 상태를 닫힘으로 되돌린 뒤에도 확인 창은 남아 있어 직접 “확인”을 눌러 종료했다.

## 팀의 앱에서는 이렇게 활용할 수 있다

- **출발 전 점검:** 문·창문·프렁크·탑승 상태를 묶은 체크리스트.
- **가족 탑승 시나리오:** 좌석 상태와 도어 이벤트에 따른 안내. 좌석 카드는 확인했지만 이번에는 직접 변경하지 않았다.
- **경고 후 복귀:** 상태 정상화뿐 아니라 남아 있는 팝업·확인 UI도 함께 시험.

창문 50%는 콘솔의 상태 반영을 확인했다. 유리 위치의 정확한 3D 애니메이션 비율까지 계측한 것은 아니다. 차량 기본 경고 UI를 우리 앱이 임의로 대체할 수 있다는 뜻도 아니다.

# 공모전 MVP는 어디서 시작할까

| 기능 후보 | 개발 접근 | 확인해야 할 조건 |
|---|---|---|
| 차량 상태 요약 | Vehicle SDK로 상태 조회·변경 구독 | 기능별 capability, 읽기 권한, 미지원·실패 처리 |
| 출발 전 점검 | 주차 상태에서 도어·창문·에너지 상태를 모아 안내 | 읽을 수 있는 실제 API, 상태 신선도, 주행 전환 시 UI 축소 |
| 운행 상황별 추천 | 기어·주행 상태·에너지 조건으로 안내 선택 | 주행 중 허용 UI와 실제 restriction 값 |
| 내비 연계 안내 | NaviHelper 연동 후보 | 내비 탑재 구성과 제공 API; 이번 실험에서는 미검증 |
| 공조·도어 자동 제어 | Vehicle 제어 API 후보 | WRITE 지원, CONTROL 권한, 제공·승인 범위부터 확인 |
| 음성·AI 앱 동작 | Gleo App Schema → Action → Callback [S8] | 연동 절차와 사용 가능 환경; 음성 호출은 미검증 |

## SDK 사용 가능 상태는 별도 관문이다

조회한 프로젝트 생성·SDK 신청 가이드는 **Vehicle State와 NaviHelper를 신청 없이 사용 가능**으로 안내한다. 반면 **Vehicle Control, FusedLocation, TextToSpeech, SpeechToText는 "준비 중"**으로 설명한다. 문서에 API가 존재하거나 에뮬레이터에 서비스가 설치돼 있어도, 우리 프로젝트에서 사용할 수 있다고 단정하면 안 된다. [S6]

> **문서의 "호환성"과 "신청 가능"은 다른 의미다.**
> 다른 문서에는 TTS/STT가 호환된다고 안내되어 있다. 이는 SDK 신청 상태나 참가자별 권한을 보장하지 않는다. 아이디어 확정 전 공모전 제공 권한과 Playground 프로젝트의 실제 상태를 확인한다. [S5, S6, S7]

## 권장 개발 순서

1. 읽기 중심의 핵심 기능 하나를 정하고 필요한 API·권한을 확인한다.
2. capability 확인 → 현재값 조회 → 변경 구독 → 오류·해제 처리를 구현한다.
3. P / D·정지 / 실제 이동 / P 복귀를 같은 데모에서 시험한다.
4. 제어·음성·위치 연동은 제공 상태를 확인한 뒤 확장한다.

# 주행 제한을 앱 설계에 반영하는 방법

- **속도만 보지 않는다.** 플랫폼의 `CarUxRestrictionsManager`를 통해 현재값과 변경 이벤트를 받는다. [S3]
- **DO 선언은 면제권이 아니다.** DO 앱도 `ActiveRestrictions`에 맞춰 영상·설정 등 개별 기능을 제한해야 한다. [S3, S4]
- **지원·권한·영역을 함께 확인한다.** capability가 있어도 필요한 권한과 승인 범위가 없으면 사용할 수 없다. [S5]
- **에뮬레이터와 실차를 구분한다.** 지역·차종·플랫폼 버전, 서비스 가용성에 따라 조건이 달라진다. [S4, S7]

## 최소 재현 순서

1. `Pleos_Connect_2_1_0` AVD를 실행하고 ADB 연결을 확인한다.
2. 8088 포트를 전달하고 콘솔에서 `DDS ONLINE`을 확인한다.
3. P에서 일반 앱을 연 뒤 D로 변경한다. 0km/h와 이동 중을 각각 기록한다.
4. ACCEL을 눌렀다가 놓고 BRAKE로 정지시킨다. 정지와 P 복귀를 따로 시험한다.
5. Emulation Controls에서 필요한 기능의 영역·값을 바꾸고 차량 화면과 앱을 확인한다.
6. 마지막에 P로 복귀하고 변경했던 값·열린 경고 창을 복원한다.

```
adb devices
adb -s emulator-5554 forward tcp:8088 tcp:8088
adb -s emulator-5554 shell am start -W \
  -n dev.pleos.starter/.MainActivity
adb -s emulator-5554 shell dumpsys car_service
```

## 이번 실험에서 확인한 한계

| 항목 | 관찰과 문서화한 범위 |
|---|---|
| 런타임 안정성 | host 렌더러와 SwiftShader 실행에서 각각 비정상 종료를 관찰했다. 이후 host + Vulkan 비활성화 + 카메라 제외 조건으로 나머지 캡처를 진행했다. 장시간 안정성이 입증된 것은 아니다. |
| 그래픽 | SwiftShader에서는 일부 차량 3D 표시가 비어 있었다. 후반 캡처는 host 조건에서 3D 차량을 확인했다. 브라우저 Highway는 WebGL 2 오류, Forest는 정상 표시였다. |
| ADB 문서와 이미지 차이 | `hmg_car_service`는 이 이미지에 없었다. `cmd car_service help`는 non-user build 요구 오류가 났다. 권한을 우회하지 않고 공식 콘솔과 허용된 진단만 사용했다. [S9] |

# 직접 관찰과 공식 문서를 함께 사용했다

사진은 이 보고서를 위해 실제 에뮬레이터와 로컬 Virtual Console에서 직접 캡처했다. 합성 화면을 사용하지 않았다. 사진의 기기 시각은 호스트 시각과 다를 수 있으며, 날짜는 호스트의 실험일을 기준으로 했다.

1. **[S1] Pleos Connect Virtual Console 사용 가이드** — 도메인·카드·설정 범위
   `https://document.pleos.ai/docs/pleos-only/app-test-guide/virtual-console-guide`
2. **[S2] Pleos Connect Virtual Console 셋업 가이드** — 접속, AVD, 테마·시점
   `https://document.pleos.ai/docs/connect/guide/getting-started/application-development/use-vehicle-emulator`
3. **[S3] 주행 규제 개발 가이드** — CarUxRestrictions, DO, 앱 구현 책임
   `https://document.pleos.ai/docs/pleos-only/vehicle-app-developer-guide/driving-restriction-development-guide`
4. **[S4] 지역별 주행 규제 정책** — Parking / Idling / Driving, 지역·차량별 차이
   `https://document.pleos.ai/docs/pleos-only/vehicle-app-planning-guide/driving-restiction-policy`
5. **[S5] SDK Overview / Capability / Permission** — 지원·접근·승인 범위
   `https://document.pleos.ai/docs/pleos-only/connect-sdk-pleos/`
   `https://document.pleos.ai/docs/pleos-only/connect-sdk-pleos/Vehicle/check-capability`
   `https://document.pleos.ai/docs/pleos-only/connect-sdk-pleos/Vehicle/vehicle-permission`
6. **[S6] 프로젝트 생성 및 SDK 사용 신청** — SDK별 사용 가능·준비 중 상태
   `https://document.pleos.ai/docs/pleos-only/create-my-project`
7. **[S7] API 호환성 및 가용성** — 차종·내비 탑재·위치 지원 조건
   `https://document.pleos.ai/docs/pleos-only/vehicle-app-planning-guide/api-compatibility-availability`
8. **[S8] Gleo Interface** — App Schema / Action / Callback
   `https://document.pleos.ai/docs/pleos-only/app-integration-guide/guide/core-concepts`
9. **[S9] ADB 디버깅 가이드** — 명령의 실제 이미지 적용 여부 대조
   `https://document.pleos.ai/docs/pleos-only/vehicle-app-developer-guide/adb-command-reference-guide`

## 공유·재검토용 파일

최종 PDF와 함께 같은 디렉터리의 `guide.html` 및 `assets/`에 원본 캡처가 있다. 주요 비교는 `04-driving-starter`, `06-driving-relaunch`, `12-energy-native`, `14-door-open-native`, `22-parked-ready` 이미지다.

> **팀 의사결정 기준**
> “콘솔에서 시뮬레이션 가능” → “우리 앱에서 읽기 가능” → “우리 앱에서 제어 가능” → “배포·실차에서 사용 가능”을 한 단계로 묶지 않는다. 각 단계의 근거가 확보된 기능부터 MVP에 넣는다.